

Vibe-IC — ALL Steps, by Stage, in Order (v2.2.0)

Plugin 0.2.6. ONE list. Every step of the flow, grouped by stage, in execution order, with a single continuous number (1 → 38). Two parallel tracks (Analog, Mixed-signal) are listed after the main flow. Source of truth = the runners.

The runner's internal implementation markers (Phase 1 [1/15] ... [15/15] + 19 sub-steps; Phase 2/3 `def step_*`) are a finer-grained code view, auto-generated in **FLOW_STEPS_GENERATED.md** by `flow_doc_emit.py`. They map onto the stages below; this doc is the single human-facing ordered list.

Stage map: **0** Spec/Docs (Phase 1) • **1** RTL+Verify • **2** Synthesis+DFT • **3** Physical+Sign-off (Phase 3) • **4** Output+Tapeout. (Phases: Phase 1 = Stage 0; Phase 2 ≈ Stage 1-2; Phase 3 ≈ Stage 3-4.)

Stage 0 — Spec & Documents (Phase 1)

#	Step	Tool / How
1	Ingest & text extraction (prompt or vendor docs → <code>input_doc/</code>)	<code>phase1_doc_one_shot_runner</code> ([1/15])
2	Generate L1–L13 core design-layer docs	deterministic extractors ([2/15] – [14/15])
3	Generate L14–L23 protocol / timing / skeleton docs	[14b/15] – [14d/15] overlays
4	Protocol-class synthesis dispatch (81 classes)	[14e/15] – [14e3/15] (<code>is_<proto> + <proto>_synth</code>)
5	Coverage / parity report	[15/15]

Stage 1 — RTL Generation & Verification (Phase 2, front)

#	Step	Tool / How
6	Spec-to-RTL (author RTL from L-docs)	<code>spec-to-rtl</code> skill (runner WAIVES <code>step_rtl_gen</code> for <code>rtl_gen=null</code> classes)
7	Lint	<code>eda_lint</code> + hygiene gates
8	CDC / RDC check	<code>cdc-check</code>
9	Simulation	<code>testbench-gen</code> + <code>eda_simulate</code> + coverage
10	Formal verification	<code>formal-verify</code> + <code>assertion-gen</code> (informational waiver if no model)
11	FPGA early prototype	<code>eda_fpga_compile</code> / <code>eda_fpga_program</code> + on-board BIST → <code>.sof</code>

Stage 2 — Synthesis & DFT (Phase 2, back)

#	Step	Tool / How
12	Constraint setup	<code>constraint-gen</code> → <code>*.sdc</code> + 3-corner PVT
13	SDC validation	SDC lint
14	Synthesis (Yosys)	<code>eda_synth</code> + <code>synth-doctor</code> (+ tie-cell pass)
15	Pre-layout STA	<code>eda_sta_mcorner</code> (SS/TT/FF)
16	DFT insertion (scan + ATPG)	<code>dft-insert</code> + <code>atpg</code> + <code>eda_dft</code>
17	Post-DFT optimization	<code>resynth</code> / buffering

#	Step	Tool / How
18	Equivalence check (LEC)	equivalence-check + Yosys equiv

Stage 3 — Physical Design & Sign-off (Phase 3)

#	Step	Tool / How	Open-source status
19	Floorplan + PDN	eda_pnr (init)	✓
20	Clock planning	clock-planning skill	✓
21	Placement (global + detailed)	eda_pnr	✓
22	CTS	eda_pnr enable_cts=true	✓
23	Post-CTS hold fixing	repair_timing -hold	✓
24	Routing (global + detailed)	eda_pnr enable_detailed_route=true	✓
25	Parasitic extraction → SPEF	OpenRCX extract_parasitics - ext_model_file rules.openrcx.sky130A.nom.m agic	✓ FIXED v0.2.5 (real 268 KB SPEF)
26	Post-route STA (MMMC)	eda_sta_mcorner	✓
27	IR drop	OpenROAD PSM analyze_power_grid	✓ FIXED v0.2.4
28	EM check	PSM -enable_em	✓ FIXED v0.2.4
29	Antenna check	OpenROAD check_antennas	✓ FIXED v0.2.4
30	Signal integrity (crosstalk)	SPEF coupling-cap screen (Cc/ (Cc+Cg))	✓ WIRED v0.2.6 (advisory)
31	Post-layout gate-level sim (+SDF)	eda_simulate	✓
32	Physical verification (DRC / LVS / ERC + PERC-equiv)	KLayout DRC + LVS sign-off chain (below) + Magic ERC + perc_equivalent aggregate	✓ DRC/LVS/ERC ; PERC-equiv (~70% auto)
33	ECO repair loop	eco-plan	✓

Step 32 — LVS sign-off chain (under step_lvs): (1) structural LEC eda_lvs yosys_equiv → (2) device-level eda_extraction + netgen → (3) powered-netlist write_verilog -include_pwr_gnd → (4) port labels magic_port_extract_emit (Route A) / lvs_def_port_seed (Route B) → (5) **mandatory** lvs_signoff_guard (RAISES on a portless/vacuous match).

Step 32 — PERC-equivalent coverage (perc_equivalent.{rpt,json} + PERC_SIGNOFF_MEMO.md, v0.2.7-2.8): open-source stand-in for commercial Calibre PERC. AUTOMATED: antenna / IR / EM / floating-nets (verdicts read from steps 27-30/ERC). GUARDBAND: EM current-density (<0.5 mA/μm) + ≥2×2 vias. MANUAL_REVIEW (never auto-PASS, pending checklist): ESD pad-ring presence, latch-up/well-tap, cross-voltage-domain — auto-N/A for core-only macros (no pad ring) / single-supply designs. The **ESD presence** check (v0.2.8) recognises sky130 IO integral-clamp pads (gpiov2/hvc/lvc/clamped/esd) — validated on the real Caravel chip_io.def (flips a 612-filler ring from a false ESD_MISSING to PRESENT, structural fillers excluded) with negation guards (unclamped ≠ ESD) + esd-before-structural ordering (gpiov2_corner_pad=ESD), and reports esd_presence: PRESENT | MISSING | N/A.

Stage 4 — Output & Tapeout (Phase 3, close)

#	Step	Tool / How	Open-source status
34	Power analysis	pre + post layout	✓
35	Metal fill (density fill)	OpenROAD filler_placement → filled.def	✓ FIXED v0.2.4

#	Step	Tool / How	Open-source status
36	Tapeout checklist	tapeout-checklist + signoff_audit (4/4 strict)	✓
37	GDSII output	eda_gds + def2gds (only if step 32 clean)	✓
38	FPGA final sign-off	recompile + on-board test + attestation	✓

Parallel track — Analog A1–A8 (analog_one_shot_runner.py, runs alongside Stages 1-3)

#	Step	Output
A1	spec_extract	analog/<block>/A1_spec.json
A2	topology_select	A2_topology.json
A3	netlist_gen	<block>.sp
A4	corner_sweep	A4_corners.json
A5	layout (Magic)	A5_layout.json (DRC-clean + LVS-match)
A6	post_layout_resim	A6_postsim.json
A7	hardmacro_gen	{.lef, .lib, .gds, .v} → feeds Stage 3
A8	hw_verify (HIL)	A8_hw_verify.json

Parallel track — Mixed-signal M1–M4 (skill mixed-signal-cosim, no dedicated runner)

#	Step
M1	top merge (mixed_signal_m1_top_merge_check.py)
M2	co-sim setup
M3	co-sim run
M4	integration verify

Totals

38 sequential steps (Stage 0: 1-5 • Stage 1: 6-11 • Stage 2: 12-18 • Stage 3: 19-33 • Stage 4: 34-38) + **8 Analog (A1-A8)** + **4 Mixed-signal (M1-M4)**, both parallel.

Pre-flight: P0 (mcp_server_health_check, eda_doctor). Orchestrator: vibe_ic_one_shot_runner.py runs Phase 1 → Phase 2 → Analog → Phase 3.

This is a **derived** list. The runner's live implementation markers are auto-generated in FLOW_STEPS_GENERATED.md (flow_doc_emit.py --check fails CI on drift). 繁中版見 ALL_STEPS_v2.2.0.zh-TW.md.